

EDUCORE

School Management System

Technical Documentation

[Architecture](#) · [Features](#) · [Development History](#) · [Setup](#)

Developed by

Weston Obiria

Code with Wes

codewithwes.co.ke

Laravel 12 · PHP 8.2 · Multi-Role Portals

Document date: 09 August 2026

Contents

1. Introduction
2. Ownership & Attribution
3. Development Journey
4. Goals and Scope
5. Technology Stack
6. System Architecture
7. User Roles and Portals
8. Feature Modules
9. Database Design
10. Authentication and Security
11. User Interface Design
12. Problems Encountered and Solutions
13. Teacher Portal
14. Parent Portal
15. Admin Portal Navigation
16. Database Seeding
17. Installation Guide
18. Demo Login Accounts
19. Testing Checklist
20. Known Limitations
21. Important File Paths
22. Closing Notes

1. Introduction

EduCore is a complete school management system built with Laravel. It was developed to handle everyday school operations through separate portals for administrators, teachers, students, parents, and finance staff.

The project started under the working name **Super School** and later adopted the EduCore brand in the interface. This document explains how the system is structured, what it can do, how it was built, the problems that appeared during development, and how those problems were fixed.

It is written as a technical reference for anyone installing, studying, or extending the codebase.

2. Ownership & Attribution

This system was designed and developed by **Weston Obiria**, professionally known under the brand **Code with Wes**.

Field	Detail
Developer	Weston Obiria
Brand	Code with Wes
Portfolio	codewithwes.co.ke
Project name	EduCore (Super School)
Stack	Laravel 12 / PHP 8.2

All original application design, portal structure, and documentation in this package are attributed to Weston Obiria / Code with Wes unless otherwise noted for third-party libraries (Laravel, Spatie Permission, Tailwind CSS, etc.).

3. Development Journey

EduCore was not built in one straight line. Work moved in phases. Each phase added capability, and later phases repaired issues discovered while using the system for real.

3.1 Phases

Phase	Focus	Result
1. Foundation	Users, roles, core models, admin CRUD	Basic role matrix working
2. Academic structure	Classes, streams, subjects, years, terms, students, teachers	School structure in place
3. Daily operations	Attendance, marks, homework, timetable, disciplinary	Teaching workflow available
4. Finance	Fee categories, structures, invoices, payments	Fee lifecycle available
5. Portals	Student, Parent, and Teacher dedicated interfaces	Role-specific access
6. Hardening	Foreign keys, missing routes, authorization, stream filters	Critical errors resolved
7. Polish	Monochrome UI, navigation consistency, seed data	Stable demo-ready system

3.2 What changed from day one

Early versions focused on the admin side. Over time it became clear that teachers, students, and parents needed first-class portals of their own — with scoped data, their own navigation, and clear permissions — rather than thin pages sitting on top of admin routes.

4. Goals and Scope

4.1 Goals

- Build a practical multi-role school management application in Laravel.
- Support real school workflows: structure, teaching, assessment, fees, and discipline.
- Provide working demo accounts so every portal can be explored after seeding.
- Keep the interface consistent and professional across all roles.
- Document the system clearly for study, deployment, and further development.

4.2 Out of scope in this version

- Multi-tenant hosting for many schools as a SaaS product.
- National reporting or government education APIs.
- Native mobile applications (the system is responsive web only).
- Full accounting modules such as payroll or general ledger.
- SMS or email notification gateways (can be added later).

5. Technology Stack

Layer	Choice	Notes
Language	PHP 8.2	Required by Laravel 12
Framework	Laravel 12	Routing, Eloquent, Blade, middleware
Authentication	Laravel Breeze	Session login and logout
Authorization	Spatie laravel-permission	Six application roles
Frontend	Blade + Tailwind CSS	Portal pages rendered as Blade views
Development database	SQLite	Simple local setup
Production database target	MySQL / MariaDB	Configured through Laravel .env
File storage	Laravel public disk	Homework files and attachments

6. System Architecture

EduCore follows Laravel's MVC pattern. Routes are grouped by role. Each major actor has a portal controller that loads the correct profile and only returns data that person is allowed to see.

6.1 Request flow

- A request enters through the web middleware stack (session, CSRF, authentication).
- RoleMiddleware checks that the signed-in user has the required role.
- The controller resolves the related profile (Teacher, Student, or ParentGuardian) using `user_id`.
- Eloquent relationships load only the scoped records (class, stream, linked children, assignments).
- A Blade view renders the monochrome portal layout and navigation.

6.2 Route groups

- **teacher.*** under `/teacher/...`
- **student.*** under `/student/...`
- **parent.*** under `/parent/...`
- **admin.*** under `/admin/...` for school administration features
- **/dashboard** as a smart redirect that sends each role to the correct home page

6.3 Why separate portal controllers matter

If teachers only used admin controllers, permissions and user experience became confusing. Dedicated portal controllers keep teacher, parent, and student logic clear, safer, and easier to maintain.

7. User Roles and Portals

Role	Main responsibilities
Super Admin	Full control of the system, staff accounts, and all modules
Admin	Academic structure, people, attendance, disciplinary records, announcements
Finance Officer	Fee setup, invoices, payments, and finance overview
Teacher	Assigned classes, homework, marks entry, and student lists
Parent	View linked children: overview, results, fees, attendance, disciplinary history
Student	Results, homework submission, attendance, fees, and timetable

7.1 Entry points

- Route **/dashboard** redirects by role after login.
- Direct portal homes exist for superadmin, admin, teacher, parent, student, and finance.
- Admin and Super Admin dashboards both allow Super Admin|Admin access so shared navigation does not cause unauthorized errors.

8. Feature Modules

8.1 People and structure

- Students with admission numbers, class and stream placement, and optional login accounts.
- Teachers with staff numbers, qualifications, class-teacher links, and subject assignments.
- Parents and guardians linked to students through a pivot table.
- Classes, streams, subjects, academic years, and terms.

8.2 Teaching and learning

- Teacher assignments connect a teacher to a subject, class, optional stream, and academic year.
- Homework supports creation, file attachments, due dates, stream targeting, student submission, and grading.
- Marks support assessment types, grading scales, and published results.
- Timetable entries store day, period, subject, teacher, and room.
- Attendance supports present, absent, late, and excused statuses with optional reasons.

8.3 Finance

- Fee categories and fee structures per class, year, and term.
- Invoices with total, paid amount, balance, and status.
- Payments recorded with method, reference, and receiving user.
- Parents and students can view fee balances and history.

8.4 Discipline and communication

- Disciplinary records store type, date, description, action taken, severity, and parent notification status.
- Announcements can target all users or specific audiences such as students, teachers, or parents.

9. Database Design

The database is relational. Students, classes, and academic years sit at the centre of most workflows. Table names are generally plural. One naming mistake around homework foreign keys caused a serious runtime error and is documented in section 12.

9.1 Important tables

Table	Purpose
users	Login accounts linked to Spatie roles
students, teachers, parent_guardians	Domain profiles, optionally linked to users
school_classes, streams, subjects	Academic structure
academic_years, terms	Calendar structure
teacher_assignments	What each teacher is allowed to teach
homeworks, homework_submissions	Assignments and submitted work
marks, assessment_types, grading_scales	Assessment and grading
attendances	Daily presence records
fee_categories, fee_structures, invoices, payments	Fees lifecycle
disciplinary_records	Behaviour incidents
timetable_entries	Weekly schedule
announcements	School messages
parent_student	Parent to child links

9.2 Access rules encoded in queries

- Students only see homework for their class, including class-wide items and their own stream.
- Parents may only open students linked through parent_student.
- Teachers may only create homework or enter marks for combinations present in teacher_assignments.
- Invoices belong to a student and may also reference a term and academic year.

10. Authentication and Security

- Users sign in through Laravel Breeze session authentication.
- Spatie roles control which route groups a user can enter.
- Portal controllers perform extra ownership checks after the role check.
- State-changing requests are protected by CSRF tokens.
- Uploaded homework files are stored through Laravel's storage system.

This level of security is suitable for demonstration, learning, and small controlled deployments. A live school environment should add stronger file validation, rate limiting, backups, and server hardening.

11. User Interface Design

The interface uses a monochrome design language: dark primary actions, light grey borders, white cards, and clear typography. Navigation is presented as pill-style links or dropdown groups depending on the portal.

11.1 Visual direction

- Primary text colours around #111 and #2a2a2a
- Muted labels around #777 to #999
- Card borders around #eaeaea
- Page background around #fafafa
- Active navigation and primary buttons around #1a1a1a
- Rounded cards with light hover elevation

11.2 Navigation by portal

- Student: Dashboard, Results, Homework, Attendance, Fees, Timetable
- Teacher: Dashboard, Classes, Students, Homework, Marks
- Parent (when viewing a child): Dashboard, Overview, Results, Fees, Attendance, Disciplinary
- Admin: Dashboard plus Academic, Finance, and More dropdown groups

12. Problems Encountered and Solutions

This section records the major technical problems found while building and testing EduCore. Each item includes the symptom, the cause, and the fix that was applied.

12.1 Homework foreign key error

What happened

When a student submitted homework, Laravel threw a database error: no such table: main.homework, even though the insert targeted homework_submissions.

Why it happened

The real table name was **homeworks**. The model used that table correctly. The migration, however, declared `foreignId('homework_id')->constrained()` without naming the related table. Laravel assumed a singular table called homework, which did not exist.

How it was fixed

- The migration was changed to `constrained('homeworks')`.
- A repair migration rebuilt homework_submissions with the correct foreign key and preserved existing rows where possible.
- Student homework listing and submission checks were aligned so class-wide and stream-specific assignments both work.

12.2 Missing parent disciplinary route

What happened

Opening a child page in the parent portal crashed with Route [parent.disciplinary] not defined.

Why it happened

The Blade navigation and the controller method already existed. The route itself had never been registered inside the parent route group in web.php.

How it was fixed

- Registered GET /parent/child/{student}/disciplinary to Parent\PortalController@disciplinary.
- Confirmed the disciplinary view and query were present.
- Included Disciplinary in the parent child navigation on all relevant pages.

12.3 Incomplete teacher routes

What happened

The teacher portal interface and controller methods for homework and marks existed, but only dashboard, classes, and students were registered as routes. Opening Homework or Marks failed.

How it was fixed

- Expanded the teacher route group with homework and marks endpoints.
- Kept server-side checks so teachers can only act on assigned subjects and classes.

12.4 Unauthorized when returning to admin dashboard

What happened

From admin subpages such as fees or classes, clicking Dashboard or the logo returned an unauthorized response.

Why it happened

Those links pointed to `superadmin.dashboard`, which only allowed the Super Admin role. Admin users were blocked.

How it was fixed

- Changed shared navigation links to `route('dashboard')`, which redirects by role.
- Allowed Super Admin and Admin on both admin and superadmin dashboard routes.
- Added the same dropdown navigation to the main dashboard so users keep context when going home.

12.5 Teacher homework not visible to students

What happened

Some assignments created by teachers did not appear in the student portal.

Why it happened

- The student was not in the class attached to the homework.
- The homework was limited to one stream while the student was in another stream or had no stream.
- Active flags or listing filters hid the item in the interface.

How it was fixed

- Student homework queries include class-wide homework and homework for the student's stream.
- Submission authorization uses the same rule set.

12.6 Unreliable demo data

What happened

The database had incomplete records and no dependable login set for every role, which made demos awkward.

How it was fixed

- `DemoDataSeeder` was rewritten with fixed demo users for all roles.
- Seed data now covers structure, assignments, fees, marks, attendance, homework, disciplinary records, timetable, and announcements.
- `DatabaseSeeder` runs roles, grading scales, assessment types, then demo data in order.

12.7 Lesson carried forward

A repeated issue was shipping views and controller methods without registering the matching routes. After any portal change, `route:list` should be checked and every navigation link should be clicked once before considering the feature finished.

13. Teacher Portal

The teacher portal is intentionally limited to what that teacher is assigned to teach. This keeps the interface focused and prevents accidental access to other classes or subjects.

- View assigned classes and students.
- Create homework for assigned subject and class combinations.
- Review submissions, download files, and enter grades with feedback.
- Enter marks using assessment types for allowed classes and subjects.
- See pending grading work on the dashboard.

14. Parent Portal

A parent signs in as a user with the Parent role and is linked to one or more students through the parent_guardians profile. Every child page checks that relationship before showing data.

- Dashboard lists linked children.
- Child overview shows fee balance, recent attendance, disciplinary summary, and recent invoices.
- Separate pages cover results, fees, attendance history, and full disciplinary history.

15. Admin Portal Navigation

Admin subpages already used a dropdown navigation model for Academic, Finance, and other areas. The main dashboard initially did not. After the unauthorized-link problem was fixed, the same navigation pattern was added to the dashboard so administrators keep a consistent path through the system.

Shared header links should always prefer a role-aware home route instead of a single privileged dashboard name.

16. Database Seeding

To reset the database and load demo data:

```
php artisan migrate:fresh --seed
```

Seed order:

- RoleSeeder creates Super Admin, Admin, Finance Officer, Teacher, Parent, and Student.
- GradingScaleSeeder creates the default A to E scale.
- AssessmentTypeSeeder creates Continuous Assessment, Mid Term Exam, and End Term Exam.
- DemoDataSeeder creates the operational demo dataset and login accounts.

16.1 Demo data includes

- Academic year 2025/2026 with three terms
- Classes and streams for Grade 7 to Form 2
- Subjects and teachers with assignments
- Students, parent links, invoices, and sample payments
- Marks, attendance, homework, disciplinary records, timetable slots, and announcements

17. Installation Guide

17.1 Requirements

- PHP 8.2 or newer with common extensions enabled
- Composer
- Optional Node.js if frontend assets need rebuilding

17.2 Setup steps

```
composer install
cp .env.example .env
php artisan key:generate
# set database settings in .env if needed
php artisan migrate:fresh --seed
php artisan storage:link
php artisan serve
```

Visit <http://127.0.0.1:8000> and sign in with a demo account. After route or view changes during development, run:

```
php artisan route:clear
php artisan view:clear
```

17.3 Useful commands

Command	Use
php artisan route:clear	Refresh routes after editing web.php
php artisan view:clear	Refresh compiled Blade views
php artisan migrate	Run pending migrations
php artisan migrate:fresh --seed	Reset database and reload demo data
php artisan storage:link	Expose public storage files

18. Demo Login Accounts

Password for all seeded accounts: **password**

Role	Email
Super Admin	superadmin@educore.test
Admin	admin@educore.test
Finance Officer	finance@educore.test
Teacher	teacher1@educore.test
Teacher 2	teacher2@educore.test
Parent	parent1@educore.test
Student	student1@educore.test

A useful walkthrough order is: Admin overview, Teacher creates homework, Student submits, Teacher grades, Parent reviews the child record, Finance reviews invoices.

19. Testing Checklist

19.1 Authentication and home pages

- Each role can sign in and reach a sensible home page.
- route('dashboard') works for Admin and Super Admin without unauthorized errors.
- Dashboard and logo links from admin subpages return home correctly.

19.2 Teacher

- Only assigned classes and subjects appear when creating homework.
- Matching students can see the homework.
- Submission, download, and grading work end to end.
- Marks entry is limited to allowed combinations.

19.3 Student and parent

- Student can list and submit homework.
- Student results, fees, attendance, and timetable pages load with navigation.
- Parent can open only linked children.
- Parent disciplinary page resolves without missing-route errors.

20. Known Limitations

- Many portal pages are full standalone Blade documents. A shared layout would reduce repeated markup.
- Dark mode is not implemented across the system.
- SQLite is convenient for local demos; production should use MySQL or MariaDB with backups.
- Automated feature tests are not part of the current package documentation.
- Email or SMS notifications are not connected.
- The interface is English only.
- File upload rules should be tightened further before any public school deployment.

21. Important File Paths

Path	Purpose
routes/web.php	All portal and admin routes
app/Http/Controllers/Admin/	Administration modules
app/Http/Controllers/Teacher/PortalController.php	Teacher portal logic
app/Http/Controllers/Student/PortalController.php	Student portal logic
app/Http/Controllers/Parent/PortalController.php	Parent portal logic
app/Models/	Eloquent models
database/migrations/	Schema definitions and repairs
database/seeders/	Roles, grading, assessment types, demo data
resources/views/admin/	Admin interface
resources/views/teacher/	Teacher interface
resources/views/student/	Student interface
resources/views/parent/	Parent interface
resources/views/dashboards/superadmin.blade.php	Main administration dashboard

22. Closing Notes

EduCore was built as a practical multi-role school system and improved by confronting real failures: incorrect foreign keys, missing routes, wrong dashboard targets, incomplete teacher and parent workflows, and weak demo data.

Those problems led to clearer portal boundaries, stricter route registration, role-aware navigation, and a reliable seed set. The system is now structured enough to install, study, demonstrate across roles, and extend with confidence.

Future work that would help most is shared Blade layouts, automated tests, and production database practice. The expensive lessons around routing completeness and foreign-key naming are already documented here so they do not need to be rediscovered.

EduCore School Management System

Developed by Weston Obiria · Code with Wes · codewithwes.co.ke

End of documentation